

APACHE DORIS 3.0.8

复习题答案

第一章：定位、架构与设计思想

10 道章末复习题的参考答案。每题包含一句话结论、底层因果链、得分关键词与常见误区。

配套资料：notes-01

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航

这不是背诵稿。建议先独立回答，再用答案中的因果链检查自己是否只记住了名词，却没有解释清楚“为什么”。

每题可按 10 分自测：结论准确 2 分；关键机制 4 分；能串起因果链 2 分；能指出边界或反例 2 分。

题号	核心问题	应答关键词
1	MySQL 多个只读副本为何不等于 Doris	单查询并行、列存、MPP、OLTP/OLAP
2	FE 为何不能承载全部 Shuffle	控制面、数据面、瓶颈、BE Exchange
3	五个存储层级分别是什么	Partition、Tablet、Replica、Rowset、Segment
4	列存、向量化、Pipeline、MPP 的区别	布局、算子、调度、分布式
5	Runtime Filter 为何同时省三类成本	下推、少扫描、少 Shuffle、少 Join
6	COMMITTED 与 VISIBLE 的区别	提交、版本发布、一致可见
7	实时数仓为何常用微批	摊薄事务、Rowset、版本、Compaction
8	何时用 Flink 而不是 Routine Load	事件时间、状态、乱序、复杂流处理
9	存算分离扩容快但首查为何抖	共享主数据、空缓存、远端读、预热
10	如何选 Local Mode 或 Cloud Mode	时延、弹性、隔离、成本、运维能力

答题方法

- 先给结论，再说机制；不要一上来堆术语。
- 把每个性能结论写成链路：少读数据 → 少网络 → 少计算 → 延迟下降。
- 谈架构选择时必须说约束；没有脱离业务条件的绝对最优。
- 涉及 3.0 存算分离时，要区分共享主数据与 BE 本地 File Cache。

1. 为什么给 MySQL 增加多个只读副本，仍然不等于得到 Doris？

只读副本主要扩展的是查询并发和可用性；Doris 还通过列式存储、数据分片、单查询跨节点并行、向量化与 MPP，改变了重型分析查询的执行方式。

原理链路

MySQL 主从复制把一份近似相同的数据放到多个实例。路由层可以把不同查询分散到不同副本，因此吞吐量会上升；但一条扫描数十亿行的 SQL 通常仍由一个实例完成，它不会因为有三个只读副本就自动拆成三份并行扫描。

维度	MySQL 只读副本	Doris
主要目的	读写分离、容灾、扩展并发	大规模分析与单查询并行
典型存储	面向事务的行式组织	面向分析的列式 Segment
一次重查询	通常落到一个实例	按 Tablet 和 Fragment 分发到多个 BE
数据削减	依赖 B+Tree 等事务型索引	分区/分桶裁剪、ZoneMap、Bloom、列裁剪
执行方式	偏点查、短事务、单机执行	向量化、Pipeline、MPP、局部聚合

完整答案应该包含

1. 副本能够把不同查询分摊出去，但不等于把一条重查询拆开。
2. 行存与列存的 I/O 单位不同；分析只读少量列时，列存通常少读很多字节。
3. Doris 把表切成 Tablet，让多个 BE 对同一条 SQL 并行扫描、Join 和聚合。
4. 二者应协作：MySQL 保持业务交易，CDC 把变化同步到 Doris 做分析。

得分关键词

读副本扩并发；单查询并行；列存；Tablet；向量化；MPP；OLTP 与 OLAP 分工

常见误区：回答“因为 Doris 机器更多”。机器数量不是根因；如果执行模型仍是单机重扫，机器再多也不会自动加速同一条 SQL。

2. FE 为什么不能承担 BE 之间的所有 Shuffle 数据？

FE 是控制面，负责接入、元数据、优化与调度；大规模中间数据属于数据面，应由 BE 之间直接 Exchange。否则 FE 会成为网络、内存和故障域的集中瓶颈。

如果所有 Shuffle 都经过 FE

错误的数据路径



- 网络瓶颈：所有 BE 的中间结果集中占用 FE 网卡。

- 内存瓶颈：FE 需要缓冲大量数据，元数据与优化任务会被挤压。
- 扩展失效：增加 BE 后，中间流量更大，但 FE 吞吐并未同步增加。
- 故障放大：FE 抖动会同时影响查询控制和数据传输。

正确分工

FE 把物理计划切成 PlanFragment 并下发；BE 按分区键对中间数据做 Hash Shuffle、Broadcast 或 Gather。局部扫描和局部聚合尽可能先在数据所在 BE 完成，再只交换必要结果。最终少量结果由 FE 返回客户端。

得分关键词

FE=控制面；BE=数据面；PlanFragment；Exchange；去中心化数据路径；避免单点瓶颈

常见误区：“FE 完全不接触查询结果”也不准确。FE 会协调查询并返回最终结果，但不应成为所有大规模中间数据的必经中转站。

3. Partition、Tablet、Replica、Rowset、Segment 分别是什么？

它们不是五个并列名词，而是一条从逻辑管理到物理文件的层级链：分区负责生命周期和裁剪，Tablet 负责水平分片，Replica 负责经典模式下的可用性，Rowset 表示版本数据集，Segment 承载真实列数据与索引。

存算一体中的典型层级



层级	定义	主要解决的问题
Partition	按日期或业务范围划分表的逻辑区域	分区裁剪、冷热管理、生命周期
Tablet	Partition 经分桶后的水平数据分片	调度、迁移、并行扫描与负载均衡
Replica	经典存算一体中 Tablet 在不同 BE 的物理副本	容灾、仲裁与高可用
Rowset	一次导入或 Compaction 形成的版本数据集	表示某段版本范围的数据
Segment	Rowset 内不可变的列式数据文件	存放 Column、Page、编码、压缩与索引

必须补充的两个边界

- Tablet 是数据管理和调度的关键单位，不等于一个物理文件；一个 Tablet 可包含多个 Rowset，每个 Rowset 又可包含多个 Segment。
- 存算分离中主数据位于共享存储，BE 主要保留 File Cache；因此不要机械套用“每个 Tablet 必须有多个本地持久副本”的经典图景。

得分关键词

Partition=范围；Tablet=分桶后分片；Replica=经典本地副本；Rowset=版本集合；Segment=列式文件

常见误区：把 Partition 说成物理副本，或把 Tablet 说成 Segment 文件，都会混淆逻辑切分、管理单位和物理文件三个层次。

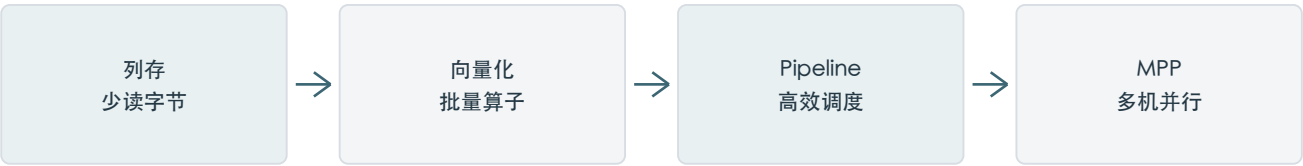
4. 列式存储、向量化、Pipeline 和 MPP 的区别是什么？

四者分别作用在不同层：列存决定数据如何落盘，向量化决定算子一次处理多少行，Pipeline 决定任务如何调度和衔接，MPP 决定一条查询如何跨多机并行。

机制	所在层次	核心收益	不负责什么
列式存储	存储布局	只读所需列、同类数据易压缩、索引易下推	不决定线程如何调度
向量化	算子实现	批量处理，减少函数调用并利用 SIMD	不决定数据放在哪台机器
Pipeline	单节点执行调度	算子流水、任务切分、并行与等待解耦	不等于多机分布式
MPP	集群执行	一条 SQL 拆到多个 BE 并行，局部计算后汇总	不替代列存与向量化

把它们串成一条查询链

四层优化叠加，而非同义词



查询先利用列裁剪和存储索引缩小扫描范围；BE 的 Scan、Filter、Join、Aggregate 以列批为单位执行；Pipeline 把 Fragment 拆成可调度任务，减少阻塞与线程空转；多个 BE 再通过 MPP 并行处理各自 Tablet，并用 Exchange 交换必要数据。

得分关键词	列存=布局；向量化=批量算子；Pipeline=节点内调度；MPP=跨节点并行；四者叠加
-------	--

常见误区：把“向量化”解释成“多机并行”，或把“Pipeline”解释成“列式存储”，说明层次没有分清。

5. 为什么 Runtime Filter 能同时减少 I/O、Join 和网络成本？

Join 的小表一侧先生成 IN、Bloom 或 Min/Max 等过滤器，并下推到大表 Scan；无匹配可能的数据在读取早期就被排除，因此后续扫描、Shuffle 和 Join 输入都会变小。

过滤越早，后续算子的收益越大



三类成本是怎样一起下降的

1. I/O：过滤器参与分区、Segment、Page 或行级判断，减少需要读取和解码的数据。
2. 网络：被过滤掉的 Probe 行不再参与跨 BE Shuffle 或 Broadcast 后的探测。
3. Join：Hash Table 探测次数、Join 中间结果、CPU 与内存消耗随输入行数下降。

为什么不是永远有效

- 过滤器生成得太晚，Scan 已经输出大量数据。
- Build 侧没有选择性，过滤器几乎允许所有值通过。
- Bloom Filter 允许假阳性；它能保证过滤掉的一定不匹配，但通过的不一定匹配。
- 过滤器的生成、合并和传输也有成本，优化器会裁剪收益不足的 Runtime Filter。

得分关键词

Build/Probe；IN/Bloom/MinMax；Scan 下推；少 I/O；少 Shuffle；少 Join 探测；Try-best

常见误区：Runtime Filter 不是预先创建的永久索引。它是在查询执行过程中根据 Join Build 侧数据动态生成的。

6. COMMITTED 与 VISIBLE 的区别是什么？

COMMITTED 表示事务已经提交，但新版本可能仍在发布；VISIBLE 表示版本发布完成，之后开始的新查询可以读取这批数据。

导入事务的可见性链路



为什么提交后还要发布版本

一次导入可能同时写入多个 Tablet。事务提交说明协调层已经接受这次提交，但各 Tablet 还需要完成版本发布，使同一批数据以一致的快照参与查询。把“文件写完”“事务提交”和“查询可见”分开，才能在分布式环境中避免查询读到半批数据。

状态	能说明什么	查询是否应读到新数据
COMMITTED	事务已提交，等待或正在发布版本	不能仅凭此状态保证可见
VISIBLE	相关版本已经发布完成	之后开始的新查询可以读取

与查询快照的关系

Doris 的查询语句开始时获取稳定快照。即使某次导入在语句执行期间变为 VISIBLE，正在运行的语句也不会突然混入这批新数据；下一条查询才会依据新的可见版本取数。

得分关键词	COMMITTED=提交完成；VISIBLE=版本发布完成；多 Tablet 原子可见；语句级稳定快照
-------	---

常见误区：“COMMITTED 就一定能立刻查到”是错误的。监控数据新鲜度时，应关注最终是否到达 VISIBLE，而不只是导入请求是否返回提交成功。

7. 为什么实时数仓通常采用微批而不是逐行事务？

微批把短时间窗口内的多行合并为一次分布式提交，摊薄事务、网络、版本发布和文件生成成本，同时把延迟控制在秒级或更低的可接受范围。

影响	逐行事务	微批 / Group Commit
事务与网络	每行都支付协调、日志与往返成本	多行共享一次提交成本
存储文件	容易生成大量小 Rowset/Segment	形成更适合扫描和压缩的数据块
版本数量	版本快速增长，发布压力高	版本增速更可控
Compaction	小文件合并压力大，易追不上写入	后台合并更稳定
延迟	单行看似低，但系统抖动容易放大	用很短窗口换取稳定吞吐

实时不等于每一行都独立提交

实时数仓关心的是端到端新鲜度：源端变化、消息传输、Doris 导入、版本变为 VISIBLE，再到查询返回。若逐行提交让 Compaction、版本发布和 FE/BE 调度过载，尾延迟反而会更差。微批是在新鲜度与吞吐稳定性之间取得工程平衡。

得分关键词	摊薄事务；减少网络往返；减少小 Rowset；控制版本数；降低 Compaction 压力；端到端延迟
-------	---

常见误区：微批不是传统离线大批。它通常是毫秒到秒级的小窗口，目标仍然是持续写入和快速可见。

8. 什么时候应把复杂处理放在 Flink，而不是 Doris Routine Load？

Routine Load 适合从 Kafka 持续消费并做轻量映射后直接入表；出现事件时间窗口、Watermark、乱序处理、CEP、长期状态、复杂去重或多流 Join 时，应先用 Flink 计算，再把结果写入 Doris。

需求	Routine Load	Flink
持续消费 Kafka	适合，内置任务与 Offset 管理	也能完成，但链路更重

需求	Routine Load	Flink
字段映射/过滤	适合轻量转换	可以，但通常没有必要
事件时间窗口	不擅长	Watermark 与窗口语义
乱序/迟到数据	不擅长复杂规则	状态化处理与补偿
CEP/长期状态	不适合	适合复杂事件和状态机
多流 Join/复杂去重	能力有限	适合有状态关联与去重
结果长期分析	负责写入 Doris	计算后仍可写入 Doris 服务查询

判断原则

- 只是把 Kafka 的 CSV/JSON 持续装进一张 Doris 表：优先 Routine Load。
- 需要理解事件发生时间、等待迟到事件或维护跨事件状态：优先 Flink。
- 复杂逻辑可以在查询时完成且无需预计算：保留明细进入 Doris，再用 SQL 或物化视图。
- 不要为了“架构看起来完整”而强行加入 Flink；每增加一层都会增加状态、一致性和运维成本。

得分关键词

Routine Load=持续导入；Flink=事件时间/Watermark/状态/乱序/CEP/多流 Join；按复杂度分工

常见误区：“有 Kafka 就必须有 Flink”是错误的。是否需要 Flink，取决于流计算语义，而不是消息队列本身。

9. 为什么存算分离扩容更快，却仍可能出现首查抖动？

新计算节点无需迁移 Tablet 主数据，连接共享存储即可工作，所以扩容更快；但它的本地 File Cache 为空，第一次查询可能从对象存储远程拉取数据，延迟和网络开销都会上升。

弹性消除了数据搬迁前置条件，但没有消除缓存预热



为什么首查更容易抖

- 对象存储的单次访问延迟通常高于本地 SSD。
- 高并发冷读会受远端存储 QPS、带宽和网络波动影响。
- 多个新节点同时取热点数据，可能形成短时下载峰值。
- 缓存命中率尚未稳定，查询之间的远端读比例差异较大。

如何缓解

- 按热点表或分区做 Cache Warmup，并观察缓存命中率。
- 为 File Cache 预留足够本地 SSD、合理容量和淘汰策略。

- 扩容后先小流量预热，再逐步承接高并发查询。
- 把冷数据扫描与延迟敏感的热点查询放到不同计算组。

得分关键词	共享主数据；无状态计算节点；免迁移；File Cache；Cache Miss；对象存储；预热
-------	---

常见误区：“无状态 BE 不需要磁盘”是错误的。它不持久化唯一主数据，但本地 SSD 对 File Cache、稳定查询延迟和远端成本非常重要。

10. 你的业务更适合 Local Mode 还是 Cloud Mode？决定因素是什么？

优先存算一体：负载稳定、有本地 SSD、追求可预测低延迟和简单运维；优先评估存算分离：计算/存储需独立伸缩、多团队要物理隔离、冷数据多、运行在云或 Kubernetes，并且有可靠共享存储与平台运维能力。

决策维度	倾向 Local Mode	倾向 Cloud Mode
性能目标	本地路径、低延迟、稳定可预测	接受缓存命中与远端存储影响
负载形态	长期稳定、扩容容频率低	波峰波谷明显、需要快速弹性
资源关系	计算和存储大致同比增长	计算和存储增长速度不同
团队隔离	逻辑隔离即可	多计算组做物理工作负载隔离
数据温度	热点占比高、主要查近期数据	冷数据多、共享低成本存储更有价值
基础设施	本地机房、可靠 SSD	对象存储/HDFS、云或 Kubernetes
运维能力	希望组件少、路径直接	能维护 Meta Service、FoundationDB、缓存和共享存储
成本结构	接受本地多副本 SSD	关注存储成本、资源利用率与弹性

推荐的决策顺序

1. 先问是否有可靠共享存储和平台团队；没有时，不要只因“云原生”选择存算分离。
2. 再看负载是否需要独立弹性和物理隔离；这是 Cloud Mode 最有价值的地方。
3. 最后用真实查询验证缓存命中、首查延迟、远端带宽和成本，不能只比较理论架构。
4. Doris 3.0.8 的部署模式应在集群建设前确定；不要把它当作运行期可随意切换的开关。

一个简短结论

如果你的答案只是“Cloud Mode 更先进”，说明还没有做架构决策。真正的答案应同时包含工作负载、SLA、资源弹性、数据温度、共享存储、运维成熟度和成本模型。

得分关键词	Local=本地路径/稳定/简单；Cloud=弹性/隔离/共享存储；SLA；缓存；成本；运维能力
-------	--

常见误区：不存在无条件更快的模式。存算一体优化本地路径与可预测性；存算分离优化资源弹性、隔离和存储成本。

答案速记

题号	一句话记忆
1	只读副本扩查询并发，Doris 还能让一条 SQL 跨节点并行。
2	FE 管控制，BE 跑数据；大 Shuffle 必须去中心化。
3	Partition → Tablet → Replica → Rowset → Segment：范围、分片、副本、版本、文件。
4	列存管布局，向量化管算子，Pipeline 管调度，MPP 管多机。
5	过滤越早，扫描、网络和 Join 输入就越小。
6	COMMITTED 是已提交，VISIBLE 才是新查询可读。
7	微批用极短等待换取事务摊薄和系统稳定。
8	轻导入用 Routine Load；复杂事件时间与状态计算用 Flink。
9	共享主数据让扩容免搬迁，空缓存让首查仍可能远端读。
10	Local 选本地低延迟，Cloud 选弹性隔离；最终由约束决定。

参考资料

答案依据第一章内容整理，并对以下 Apache Doris 官方文档中的关键机制进行了交叉校验。

- [1] System Architecture
<https://doris.apache.org/docs/3.x/features-architecture/system-architecture/>
- [2] Data Distribution
<https://doris.apache.org/docs/3.x/table-design/data-partitioning/data-distribution/>
- [3] Runtime Filter
<https://doris.apache.org/docs/3.x/query-acceleration/optimization-technology-principle/runtime-filter/>
- [4] Transaction
<https://doris.apache.org/docs/3.x/data-operate/transaction/>
- [5] Loading Overview
<https://doris.apache.org/docs/3.x/data-operate/import/load-manual/>
- [6] Routine Load
<https://doris.apache.org/docs/3.x/data-operate/import/import-way/routine-load-manual/>
- [7] Flink Doris Connector
<https://doris.apache.org/docs/3.x/ecosystem/flink-doris-connector/>
- [8] Compute-Storage Decoupled
<https://doris.apache.org/docs/3.x/compute-storage-decoupled/overview/>
- [9] File Cache
<https://doris.apache.org/docs/3.x/compute-storage-decoupled/file-cache/>